

JAVA INTERVIEW PREPARATION

CHAPTER 19

ARRAYLIST AND LIST INTERNALS

Senior / Lead Java Developer Textbook Edition

Full reading material - not summarized

ArrayList and List Internals: Contiguous Storage, Growth, Views, and Workload Fit

Senior / Lead Java Developer Reading Chapter

ArrayList is the default general-purpose List for good reason: it is compact, cache-friendly, and provides fast indexed access. Its apparent simplicity can hide costly insertion patterns, repeated growth, retained capacity, boxing overhead, and subtle view behavior. Senior-level selection begins with the workload rather than the interface name.

The Internal Model

An ArrayList stores element references in a contiguous Object array and maintains a logical size separately from array capacity.

```
elementData capacity = 8
index:  0    1    2    3    4    5    6    7
       [A] [B] [C] [D] [null][null][null][null]
logical size = 4
```

The list object does not contain the element objects themselves. It contains references. Adding a large object adds one reference to the backing array; the large object lives elsewhere on the heap.

`get(index)` checks the index and directly accesses the array slot, so it is constant time. Modern CPUs also benefit from the contiguous reference array when iteration proceeds in order.

Size Is Not Capacity

Size is the number of list elements exposed by the API. Capacity is the backing array length. Capacity may exceed size so future appends do not allocate on every call.

```
List<Order> orders = new ArrayList<>(10_000);
System.out.println(orders.size()); // 0, not 10,000
```

Initial capacity reserves reference slots; it does not create elements. Oversizing every list wastes heap, while undersizing a known large batch causes repeated array copies.

Growth and Amortized Appends

When an append would exceed capacity, ArrayList allocates a larger array and copies existing references. OpenJDK implementations commonly grow by roughly 50 percent, but the exact growth formula is not an API contract.

```
before append: [A B C D] capacity 4
allocate:      [A B C D _ _] larger array
append:       [A B C D E _]
old array becomes garbage
```

One growth operation is $O(n)$, but a sequence of appends is amortized $O(1)$: expensive copies are spread across many cheap appends. Amortized does not mean every add is constant time. A resize on a latency-sensitive request can still be visible.

Use `ensureCapacity` before a known bulk load. Use `trimToSize` only with evidence; it copies the array and a later append may immediately grow it again.

Insertion and Removal Shift Elements

Adding at the end is normally cheap. Adding near the beginning shifts the suffix to the right. Removing by index shifts the suffix left and clears the now-unused final slot so the removed object can be collected.

```
insert X at index 1  
[A B C D _] -> [A _ B C D] -> [A X B C D]
```

The cost is $O(n)$ in the shifted portion. Repeatedly removing index zero turns a linear traversal into quadratic behavior.

```
while (!list.isEmpty()) {  
    process(list.remove(0)); // repeated full shifts  
}
```

Use `ArrayDeque` for FIFO queue behavior. Choose a collection whose structural operations match the algorithm.

The remove Overload Trap

List has both `remove(int index)` and `remove(Object value)`. With boxed integers, overload selection can surprise developers.

```
List<Integer> numbers = new ArrayList<>(List.of(10, 20, 30));  
numbers.remove(1); // removes element at index 1: 20  
numbers.remove(Integer.valueOf(10)); // removes the value 10
```

This is a compile-time overload decision, not an `ArrayList` bug. Be explicit when the element type is `Integer`.

Search Uses Equality

`contains`, `indexOf`, and `remove(Object)` scan elements and compare using equality. They are $O(n)$, and expensive equals implementations multiply that cost.

If code repeatedly asks whether an identifier is present, a `HashSet` may represent the access pattern better. Maintaining both a list and a set can be appropriate when order and fast membership are both required, but every mutation must keep them consistent.

Iterators and Structural Modification

`ArrayList` iterators are fail-fast on a best-effort basis. Structural modification outside the iterator can cause `ConcurrentModificationException`.

```
Iterator<Order> iterator = orders.iterator();  
while (iterator.hasNext()) {  
    if (iterator.next().isCancelled()) {  
        iterator.remove(); // supported iterator mutation  
    }  
}
```

Enhanced for-loops use iterators. Prefer `removeIf` for simple filtering, or build a new result when transformation is clearer. `ConcurrentModificationException` is not a concurrency control mechanism.

subList Is a Backed View

`subList` returns a view over a range of the original list, not an independent copy.

```
List<String> all = new ArrayList<>(List.of("A", "B", "C", "D"));
List<String> middle = all.subList(1, 3);
middle.set(0, "X");

System.out.println(all); // [A, X, C, D]
```

Structural modification of the backing list outside the view invalidates the view's assumptions and later operations may throw `ConcurrentModificationException`. A small long-lived `subList` can also retain the entire backing list through its parent relationship.

When an independent result is required, copy it:

```
List<String> page = new ArrayList<>(all.subList(from, to));
```

list.toArray and Generic Arrays

Because generics are erased and arrays are reified, conversions deserve care.

```
Order[] array = orders.toArray(Order[]::new);
```

The generator form clearly creates the correct runtime array type. An `Object[]` cast to `Order[]` is not valid merely because its current elements happen to be orders.

Memory and Boxing

An `ArrayList<Integer>` stores references to `Integer` objects, not packed primitive `int` values. Large numeric datasets may pay for wrappers, references, allocation, and poorer locality.

```
ArrayList<Integer>: [ref][ref][ref] -> separate Integer objects
int[]:               [int][int][int] packed values
```

For computation-heavy primitive data, arrays or a specialized primitive collection can dramatically reduce memory and improve locality. Measure before adding a third-party dependency, but understand the representation cost.

Clearing a list nulls its active references but normally retains backing-array capacity. Reusing a large list can avoid allocation; retaining it forever after a one-time spike can waste memory. Lifecycle determines the right choice.

ArrayList Versus LinkedList

`LinkedList` offers constant-time insertion or removal only after the relevant node is already available through an iterator. Finding index `n` remains $O(n)$. Each node stores element and link references, increasing allocation and reducing locality.

For most application lists, `ArrayList` is faster and smaller. `LinkedList` can fit algorithms that perform frequent iterator-local insertion/removal and do not need indexed access, but `ArrayDeque` is usually better for stack and queue operations.

```
ArrayList: compact references, fast index, shifting on middle edits
LinkedList: per-node allocation, traversal for index, cheap known-node unlink
ArrayDeque: circular array, efficient operations at both ends
```

CopyOnWriteArrayList

CopyOnWriteArrayList creates a new backing array for every structural update. Reads and iteration are simple and snapshot-like, so it fits small collections with rare writes and frequent traversal, such as listener registrations.

It is a poor choice for write-heavy data. An add to a million-element list copies a million references. Its iterator does not reflect later changes and does not support iterator removal.

Thread Safety and Publication

ArrayList is not thread-safe. Concurrent mutation can produce races, lost changes, stale reads, or inconsistent iteration. A safely published list that is never mutated can be read concurrently, but immutability must include the intended element state.

Options include external locking, synchronized wrappers with correct iteration locking, CopyOnWriteArrayList for read-mostly workloads, concurrent queues for producer-consumer flows, or immutable snapshots replaced atomically.

Production Perspective

Common incidents include repeated `remove(0)`, accidental quadratic contains loops, a `subList` retaining a huge dataset, a large backing array retained after a traffic spike, and boxed primitives multiplying heap usage. Another pattern is returning a mutable internal ArrayList from an API, allowing callers to break invariants.

Use allocation profiles, heap histograms, retained-size analysis, CPU profiles around `arraycopy` or `equals`, and realistic collection cardinalities. Complexity describes scaling; profiling shows whether the scaling matters in this service.

Interview-Ready Summary

ArrayList is a resizable Object array with separate size and capacity. Indexed access is $O(1)$, append is amortized $O(1)$, and middle insertion or removal is $O(n)$ because references shift. Growth requires allocation and copying.

`subList` is a backed view, fail-fast iteration is diagnostic, and Integer removal has an overload trap. ArrayList usually outperforms LinkedList because of locality and lower allocation. Choose ArrayDeque for queues, CopyOnWriteArrayList for small read-mostly collections, and primitive arrays or specialized collections when boxing dominates.

Revision Notes

- ArrayList stores references in a contiguous Object array.
- Logical size and backing capacity are different.
- Initial capacity does not create elements.
- get is $O(1)$; append is amortized $O(1)$.
- A growth operation allocates and copies $O(n)$ references.
- Middle insertion and removal shift a suffix.
- Repeated `remove(0)` is usually the wrong queue implementation.
- `remove(int)` and `remove(Object)` are distinct overloads.
- `contains` and `indexOf` perform linear equality searches.
- Iterators are fail-fast on a best-effort basis.
- `subList` is a backed view, not an independent copy.

- Copy a subList when independent lifetime is required.
- ArrayList<Integer> stores boxed objects and references.
- clear normally retains backing capacity.
- ArrayList is usually preferable to LinkedList.
- ArrayDeque is normally better for stack and queue workloads.
- CopyOnWriteArrayList favors many reads and very few writes.
- ArrayList requires immutability, external synchronization, or another design for concurrency.
- Profile array copying, equality scans, boxing, and retained capacity in production.